

Aplicación de EDAs a la optimización de plantillas SBC en FC24

Josep Gabriel Fornes Reynes

1 Contexto y objetivo

Este trabajo parte de un TFG previo sobre **FC24 Ultimate Team**, donde se resolvían desafíos de creación de plantillas (SBC/DCP) mediante técnicas evolutivas [1]. Cada desafío consiste en formar una plantilla de 11 jugadores que cumpla restricciones como media mínima, química, nacionalidades, ligas, clubes, número máximo de jugadores por grupo o versiones especiales de carta. Además, interesa minimizar el precio, ya que los jugadores entregados en un SBC se pierden.

En el TFG original el problema se resolvía con un algoritmo genético (GA): cada individuo era una plantilla completa; la población se evaluaba con una función de *fitness*; y se aplicaban selección, cruce, mutación y elitismo. El objetivo de este trabajo es estudiar si un algoritmo de estimación de distribuciones (EDA), concretamente UMDA categórico, puede mejorar ese baseline. Se elige UMDA porque el problema puede formularse como una selección de variables categóricas: en cada posición de la plantilla se escoge una carta. Los EDAs sustituyen los operadores clásicos de recombinación por el aprendizaje de un modelo probabilístico a partir de las mejores soluciones, desde el que después se muestrean nuevas soluciones [2, 3].

Cada solución se representa como

$$x = (x_1, \dots, x_{11}),$$

donde x_i es el jugador asignado al slot i de la formación. La función de *fitness* replica la lógica del TFG: combina desviación respecto a los requisitos, coste normalizado y una penalización adicional por cada requisito incumplido,

$$f(x) = d(x) + c(x) + \lambda u(x),$$

donde $d(x)$ mide la distancia a los requisitos, $c(x)$ el coste normalizado y $u(x)$ el número de requisitos incumplidos. Se mantiene también la regla de no repetir nombres dentro del once: el dataset conserva cartas distintas de un mismo jugador, pero una plantilla no puede contener dos veces el mismo nombre.

2 Modos de evaluación y protocolo experimental

Se conservan los dos modos del TFG: **club** y **BD**. La diferencia no está en el algoritmo, sino en la normalización del coste dentro de la *fitness*. En modo **club**, el precio se normaliza usando como referencia los jugadores disponibles en `jugadores.json` (propios del club); por tanto, el coste pesa más y se favorecen plantillas baratas. En modo **BD**, la referencia procede de la base de datos global usada en el TFG, concretamente de un máximo de precio mucho mayor precomputado en `config/price.bounds.json`; por tanto, el mismo coste queda normalizado con un valor menor y pesa menos. Así, BD no significa que el algoritmo pueda elegir cualquier jugador de la base de datos, sino que utiliza una referencia global para escalar el precio.

También se mantiene la lógica original de posiciones: un jugador puede ocupar un slot que no corresponda a su posición natural, pero entonces su química es cero. Esto es importante porque algunos SBC no requieren química alta y, en esos casos, filtrar por posición podría eliminar soluciones válidas y baratas. Sin embargo, cuando el reto exige química, la compatibilidad posicional vuelve a ser decisiva.

El protocolo usa los mismos 20 desafíos del TFG y cinco semillas por configuración, dando 100 ejecuciones por variante. La fiabilidad se calcula como requisitos cumplidos sobre el total de requisitos evaluados en esas ejecuciones (380). También se registra el coste medio de la mejor plantilla encontrada y el tiempo medio de ejecución. El GA se toma como baseline histórico y no se modifica ni en sus operadores ni en su función de *fitness*. Esta decisión evita que la comparación se desplace hacia un nuevo GA y permite centrar el trabajo en el efecto de introducir un EDA. Por ello, los tiempos no se comparan contra el GA, solo entre variantes UMDA ejecutadas en el nuevo repositorio.

3 Fase 1: sustitución directa por UMDA

La primera fase prueba una sustitución directa del GA por UMDA. Cada slot puede elegir cualquier jugador del dataset. Esta formulación es comparable con la del GA, pero resulta difícil para UMDA: hay 11 variables categóricas y cada una puede tomar 1008 valores. UMDA estima una distribución univariante,

$$P(x) \approx \prod_{i=1}^{11} P_i(x_i),$$

Modo	Algoritmo	Cumplidos	Fiabilidad	Coste medio	Runtime
club	GA histórico	322/380	84,74%	23.788,00	–
club	UMDA directo	308/380	81,05%	29.517,00	22,91s
BD	GA histórico	339/380	89,21%	77.501,50	–
BD	UMDA directo	313/380	82,37%	76.819,50	24,39s

Tabla 1: Comparación inicial entre GA y UMDA directo.

por lo que aprende probabilidades independientes por slot. El problema SBC, en cambio, no es independiente: la química depende de combinaciones de liga, club, nacionalidad y posición; la media se calcula sobre el conjunto completo; y algunas restricciones piden cardinalidades exactas o cartas especiales.

Los resultados de la Tabla 1 muestran que UMDA directo no supera al GA. En modo **club** pierde fiabilidad y además obtiene mayor coste medio. En modo **BD** el coste medio es parecido al del GA, pero la fiabilidad cae de 89,21% a 82,37%. Esto indica que el problema no era solo económico: UMDA directo no encuentra bien combinaciones factibles. El GA tolera mejor la representación amplia porque trabaja con plantillas completas y puede conservar bloques útiles mediante cruce y elitismo; UMDA, al remuestrear cada slot de forma independiente, tiende a romper esas estructuras.

4 Fase 2: UMDA estructurado

A partir del diagnóstico anterior se implementa una variante denominada `umda_structured`. La función de evaluación, los retos y los datos se mantienen, pero cambia la representación que recibe UMDA. En vez de permitir que cada slot elija entre todos los jugadores, se construye un dominio local de 150 candidatos por slot. Estos dominios priorizan jugadores compatibles con la posición, alineados con los requisitos del reto, cercanos a la media mínima y razonables en precio. También se reserva una parte para jugadores fuera de posición, de modo que la posición no se convierte en una restricción dura.

La idea no es hacer una comparación idéntica a la fase 1, sino adaptar el espacio de búsqueda al tipo de modelo que aprende UMDA. En EDAs, la representación es parte esencial del método: si el dominio categórico está demasiado disperso, las probabilidades estimadas son poco informativas; si el dominio está estructurado, el modelo puede aprender regularidades útiles con muchas menos evaluaciones.

Modo	Algoritmo	Cumplidos	Fiabilidad	Coste medio	Runtime
club	GA histórico	322/380	84,74%	23.788,00	–
club	UMDA directo	308/380	81,05%	29.517,00	22,91s
club	UMDA estructurado	342/380	90,00%	25.956,50	1,78s
BD	GA histórico	339/380	89,21%	77.501,50	–
BD	UMDA directo	313/380	82,37%	76.819,50	24,39s
BD	UMDA estructurado	348/380	91,58%	66.450,00	1,76s

Tabla 2: Resultados finales de GA, UMDA directo y UMDA estructurado.

La Tabla 2 muestra que UMDA estructurado mejora claramente. En modo **club** alcanza 342/380 requisitos cumplidos, frente a 322/380 del GA. Su coste medio es algo superior al del GA, pero inferior al de UMDA directo. En modo **BD** obtiene 348/380 requisitos cumplidos, con 91,58% de fiabilidad, y además reduce el coste medio respecto al GA. El tiempo medio baja de unos 23–24 segundos en UMDA directo a menos de 2 segundos. Esta comparación temporal solo debe hacerse entre variantes UMDA, porque no se dispone de un runtime homogéneo para el GA histórico.

5 Diagnóstico y discusión

La mejora de UMDA estructurado no resuelve todos los retos. Los fallos restantes se concentran en desafíos con media mínima alta, restricciones exactas de nacionalidad/liga o versiones escasas, como *Dynamic Duos*. En modo **club** destacan retos como *Icon Flash*, *TriNación Elite*, *Liga y Nación Mixta* o *Leyendas Supremas*. En modo **BD**, al pesar menos el coste, algunos retos mejoran, pero siguen apareciendo fallos de media, química alta o cartas especiales.

Para separar el efecto del precio de la dificultad propia de los requisitos, se ejecutó UMDA estructurado sin penalización de coste en la *fitness*. El resultado fue 350/380 requisitos cumplidos, con una fiabilidad del 92,11% y un coste medio de 113.038,50 monedas. Se resolvieron completamente 16 de los 20 retos; los cuatro restantes quedaron a un requisito pendiente: media mínima en *Icon Flash*, *Liga y Nación Mixta* y *TriNación Elite*, y dos cartas *Dynamic Duos* en *Leyendas Supremas*. Esto sugiere que parte de los errores venían del compromiso coste-requisitos, pero que también existen restricciones combinatorias difíciles con el dataset disponible.

La lectura principal es doble. Primero, UMDA directo no supera al GA porque el problema tiene dependencias fuertes entre jugadores y UMDA aprende marginales independientes. Segundo, UMDA estructurado sí mejora porque reduce

el espacio de búsqueda y aumenta la señal estadística: el algoritmo ya no debe descubrir desde cero qué jugadores son razonables para cada slot o reto. Por tanto, la conclusión no es que “UMDA gana al GA” de forma automática, sino que un EDA puede ser competitivo si el problema se codifica de forma adecuada para el aprendizaje probabilístico.

6 Limitaciones y trabajo futuro

La principal limitación es que el GA se usa como baseline histórico. No se ha reejecutado dentro del nuevo repositorio ni se ha probado con la misma representación estructurada, por lo que no se puede separar completamente el efecto del algoritmo y el efecto de la reducción del dominio. Además, los tiempos solo son comparables entre variantes UMDA. Otra limitación es que no se han probado EDAs multivariantes; modelos capaces de aprender dependencias entre slots podrían ajustarse mejor a requisitos de química, ligas y nacionalidades [3].

Como próximos pasos, sería conveniente reejecutar el GA en el nuevo entorno, aplicar dominios estructurados también al GA, probar un EDA multivariante y analizar la sensibilidad a `domain_size`, `size_gen` y `max_iter`. Estos experimentos permitirían distinguir si la mejora procede del algoritmo probabilístico, del filtrado de candidatos o de la combinación de ambos factores. Para asegurar la reproducibilidad, el repositorio conserva los datos, los CSV de resultados y los parámetros principales de ejecución.

7 Conclusión

El trabajo demuestra que aplicar EDAs al problema de optimización de plantillas SBC es viable, pero que la representación del problema es crítica. La sustitución directa del GA por UMDA no mejora el baseline, porque el modelo univariante no captura las dependencias entre jugadores. En cambio, al introducir dominios estructurados por posición y requisitos, UMDA supera al GA histórico en fiabilidad: 90,00% en modo **club** y 91,58% en modo **BD**. En BD también consigue menor coste medio. La conclusión final es que los EDAs no son automáticamente superiores a los GA, pero pueden ser muy competitivos cuando el espacio de búsqueda se formula de manera coherente con el modelo probabilístico que van a aprender.

Referencias

- [1] J. G. Fornes Reynes. *Tècniques evolutives per a la presa de decisions en videojocs*. Trabajo de Fin de Grado, Universitat de les Illes Balears, 2024–2025.
- [2] H. Mühlenbein and G. Paaß. From recombination of genes to the estimation of distributions I. Binary parameters. In *Parallel Problem Solving from Nature – PPSN IV*, pp. 178–187, 1996.
- [3] P. Larrañaga, H. Karshenas, C. Bielza and R. Santana. A review on probabilistic graphical models in evolutionary computation. *Journal of Heuristics*, 18:795–819, 2012.
- [4] P. Larrañaga. *Estimation of Distribution Algorithms*. Material del Seminario de Computación Natural, Máster Universitario en Inteligencia Artificial.